## 0.1 Quality of Service Mechanisms

The traditional Internet operates on a *best-effort* delivery model, where data packets are transmitted without any explicit guarantees on performance metrics such as latency, jitter, throughput, or packet loss. In this model, all traffic is treated equally, and no special priority is given to time-sensitive or critical applications. While this approach has enabled rapid growth and scalability of the Internet, it often falls short in meeting the stringent requirements of modern applications such as video streaming, Voice-over-IP (VoIP), online gaming, and industrial automation.

To address these limitations, many Internet Service Providers (ISPs) and network operators offer QoS as an additional feature. These services provide guaranteed performance levels for specific traffic flows or applications, allowing clients to meet their unique requirements. QoS-enabled services can include prioritization of critical traffic, dedicated bandwidth allocation, or enhanced reliability, ensuring that contractual obligations, often formalized as Service Level Agreement (SLA), are met.

The ability to purchase QoS guarantees has become increasingly important in the context of emerging technologies such as 5G networks, cloud computing, and the Internet of Things (IoT), where diverse applications demand tailored network performance.

The mechanisms underlying QoS include **classification**, **marking**, **policing**, **shaping**, and **scheduling**. This section introduces these concepts, explains their interaction, and illustrates their application in the context of Network Slicing (NS).

### 0.1.1 QoS Mechanisms

*Call Admission Control (CAC):* CAC is the first line of defense in ensuring QoS. It determines whether a new traffic flow can be admitted into the network without violating the QoS guarantees of existing traffic. By analyzing available resources (e.g., bandwidth) and current network conditions, CAC either accepts or rejects a new flow. This proactive approach prevents oversubscription and ensures that the network can meet the agreed-upon SLAs.

*Classification.* Classification identifies and groups packets based on specific attributes such as source/destination IP, port numbers, protocol, or application type. This step ensures that traffic flows are treated according to their assigned policies.

*Marking.* Marking assigns a QoS label to each packet, typically using fields in the packet header or adding a dedicated header as, e.g., in the case of Multi Protocol Label Switching (MPLS). These labels enable downstream devices to apply the appropriate QoS treatments.

*Policing.* Policing enforces bandwidth limits and ensures compliance with SLA requirements. Traffic exceeding the allowed limits are marked as *non-conformant* and may be dropped, delayed, or re-marked to a lower priority.

*Shaping.* Shaping smooths traffic bursts by buffering packets and releasing them at a controlled rate. This prevents congestion and ensures that traffic conforms to the desired transmission profile.

*Scheduling.* Scheduling determines the order in which packets are transmitted based on their priorities or weights. Mechanisms such as strict priority scheduling or Weighted Fair Queueing (WFQ) allocate resources according to QoS requirements.

These mechanisms work together in a pipeline to guarantee SLAs. The QoS chain is graphically exemplified in Fig. 0.1. The CAC operates at the entry point of the QoS pipeline, ensuring that only manageable traffic enters the network (in Fig. 0.1, the middle Information Flow is blocked). The Classification identifies traffic classes to apply specific policies (in the figure, different traffic classes are identified by colours). To be noticed that CAC can also be placed after the Classification module for a more efficient and nuanced approach to resource management, allowing the network to better enforce differentiated SLAs. The optimal placement depends on the specific network architecture and performance goals. Then, the Marking block ensures that packets carry QoS labels for consistent handling across the network (in the figure, packets have the same color of the class they belong to). Packets then enter the Policing block, which controls resource usage and enforces limits for fairness. Packets that violate the SLA can be marked as *non-conformant* and passed along the chain as *best effort* traffic (in the figure, grey packets), or dropped. The Shaping module can buffer the different packets to adjust the traffic flow to the SLA, avoid congestion and ensure smooth delivery (in the figure, the traffic peaks are levelled out). Examples of Shaping mechanisms are the leaky bucket and token bucket filters, which are explained later. Finally, the Scheduling module allocates network resources according to the service priorities (in the figure, packets are streamed into the link).

The combined effect ensures that traffic flows adhere to SLAs, providing guarantees for parameters such as latency, jitter, throughput, and packet loss. It has to be noticed, however, that not all these QoS mechanisms are always required to provide QoS guarantees: depending on the considered scenario, some mechanisms can be omitted. For example, in case the traffic sources are homogeneous in terms of type of traffic and QoS requirements, classifier and marker may not be needed. Moreover, some functionalities are implemented together, as for Shaping and Scheduling.

**Example 0.1 A** *Network slicing* in 5G networks exemplifies how QoS mechanisms are applied to create isolated virtual networks tailored to specific use cases. Each slice is configured with unique SLA requirements, such as ultra-low latency, high throughput, or massive connectivity for IoT devices. Accordingly, a network can be organized in three slices, namely Ultra-Reliable Low Latency Communication (URLLC), Enhanced Mobile Broadband (eMBB), and Machine-Type Communication (MTC).

The first slice is used for traffic identified as URLLC, whose packets are marked as prioritized for expedited forwarding. The policer limits bandwidth usage to prevent impact on other slices, but

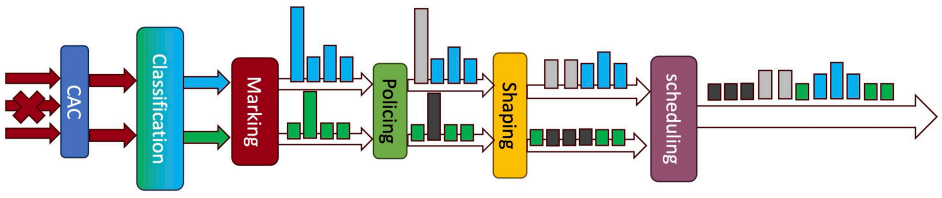



**Figure 0.1** QoS-chain.

non-conformant packets are still buffered and transmitted, though the low latency requirement may not be guaranteed for such packets. The scheduler ensures minimal latency for packets marked for expedited forwarding and conformant.

The eMBB slice is selected for streaming and multimedia flows. Policing enforces maximum throughput limits. Scheduling ensures fair resource allocation among users.

The MTC slice is chosen for traffic classified as low priority, and marked accordingly. The policer marks traffic that exceeds the agreed rate, and the shaper buffers such packets to fit the Information Flow within limited bandwidth, while tolerating higher delay. The scheduler transmits these packets using a weighted fair-share policy, but with low weight.

This layered approach ensures that network slices operate independently while coexisting on shared physical infrastructure, fulfilling the unique SLA requirements of each slice.

## 0.2 Leaky Bucket Filter model

The Leaky Bucket Filter (LBF) is a mechanism used to control data flow rates by enforcing a consistent output rate, independent of the variability or burstiness of the incoming traffic. This mechanism plays a crucial role in ensuring that traffic conforms to the desired network policies, preventing congestion and enabling predictable performance for data transmission. With reference to the QoS mechanisms of Sec. 0.1.1, the LBF can be seen as a shaper, since it buffers the traffic peaks that exceed the nominal maximum rate $\mu$ and forwards it progressively, so as to keep the output flow rate always within the SLA boundaries.

Conceptually, the LBF can be visualized as a bucket where data flows at the generation rate $\mathrm{G}(t)$ and leaks out at a maximum rate $\mu$, called *leaky rate* (see Fig. 0.2). However, the bucket has a limited capacity $b_d$, which imposes a restriction on the amount of data that can be stored at any given time. If the bucket becomes full due to a sustained high arrival rate ($\mathrm{G}(t) > \mu$ for sufficiently long time), any additional data attempting to enter is discarded.

Accordingly, the LBF can be described by two parameters: the leaky rate $\mu$ and the bucket capacity $b_d$. The behavior of the LBF can be mathematically modeled as a single-server queue with a constant service rate $\mu$ and a finite buffer size $b$. The rate of the downstream traffic can hence be expressed as

$$\mathrm{G}_{\mathrm{out}}(t) = \mathrm{G}(t)\chi\left\{b_d(t) = 0\right\} + \mu\chi\left\{b_d(t) > 0\right\} \ , \tag{0.1}$$

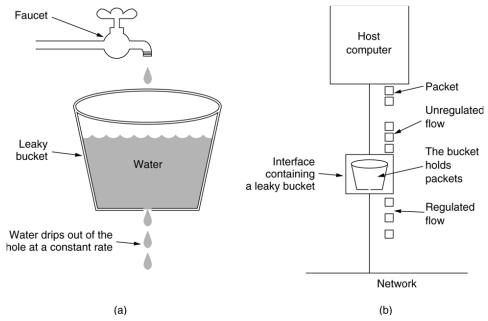



**Figure 0.2** Leaky Bucket Filter.

where $b_d(t)$ is the number of bits buffered in the bucket. To be noticed that $\mathrm{G}_{\mathrm{out}}(t) \leq \mu$ since, whenever $\mathrm{G}(t) > \mu$, $b_d(t)$ becomes positive and the first indicator random variable (r.v.) in the right-hand side of (0.1) becomes zero. As long as the bucket is not empty, the leaky rate will be equal to $\mu$.

Overall, the LBF provides a simple yet powerful method for managing traffic flows in a network, ensuring that traffic bursts are smoothed out and that data conforms to the desired rate, preventing congestion and maintaining a steady flow of packets.

## 0.3 Token Bucket Filter (TBF)

In the Token Bucket (TB) model, the bucket holds tokens, which represent permissions to transmit data. Each data transmission consumes a predefined number of tokens, typically one token per bit or fixed-size data chunk (e.g., per byte or packet). For a packet to be transmitted, the system must hold a sufficient number of tokens in the bucket. Tokens are generated at a constant rate, denoted by $\rho$ [token/s], and they accumulate in the bucket until consumed by packet transmissions. During periods of low traffic, tokens tend to accumulate in the bucket, enabling larger bursts of data to be transmitted when traffic increases. The bucket can hold up to $b$ tokens, where $b$ is the bucket size (or capacity). When the bucket reaches its capacity, any newly generated tokens are discarded. Therefore, the TB can be specified by three parameters, whose are defined in the SLA, namely:

- The *token generation rate* $\rho$, representing the average rate at which new tokens poor into the bucket and, in turn, the maximum *average* output rate of the Token Bucket Filter (TBF);
- The *peak token service rate* $\mu \geq \rho$, representing the maximum output rate allowed by TBF during bursts;
- The *bucket capacity* $b$, determining the maximum number of tokens the system can store.

A graphical representation of the TB model is shown in Fig. 0.3.

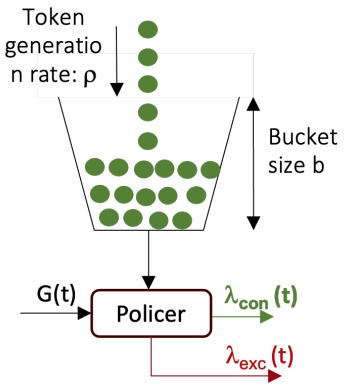



**Figure 0.3** Token Bucket filtering principle.

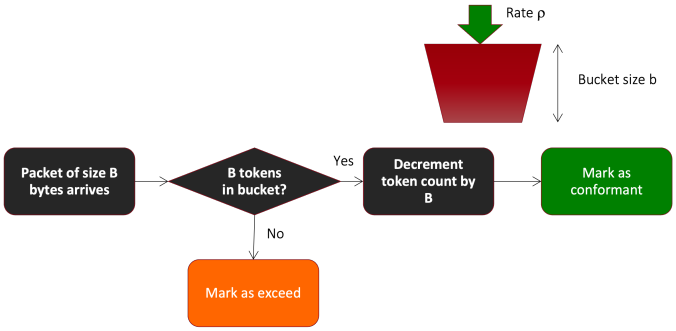



**Figure 0.4** Token Bucket Filter as a Policer.

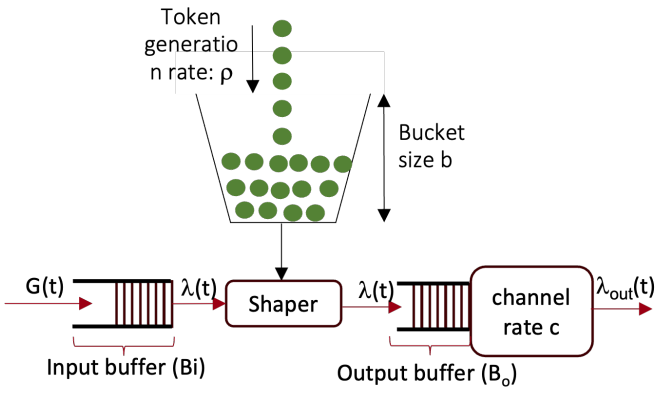



**Figure 0.5** Token Bucket Filter as a (soft) Shaper.

The TB filter then identifies as *non-conformant* (or *exceed*) the Information Units (IUs) that deviate from the desired transmission profile. When a packet arrives at the system, indeed, its transmission depends on the availability of tokens in the bucket. If the number of tokens is at least equal to the size of the packet (measured in bits, bytes or data chunks), the packet can be transmitted immediately. However, if there are insufficient tokens, the packet is *non-conformant* and the TB system must adopt a predefined policy to handle it. The possible actions are:

- Marking the packet (Policer): the packet is tagged or marked as *non-conformant* and treated accordingly (e.g., assigned to lower priority or best-effort queues). The TBF does not need an ingress buffer, since packets are not held in queue, but forwarded to the next module of the QoS chain with a non-conformant marking. In this case, the TBF is used as a Policer. A graphical representation of this functioning is given in Fig. 0.4.

- Discarding the packet (Hard Shaper): non-conformant packets are dropped and not transmitted. Again, the TBF does not need any ingress data buffer. This behavior corresponds to the a lossy traffic Shaper, since the output traffic profile is forcefully conformant to that defined by the TB parameters.

- Buffering the packet (Soft Shaper): the packet is stored in the ingress buffer and transmitted when a sufficient number of tokens are available in the bucket. Also in this case, the TB operates as a traffic Shaper, but with more graceful handling of violations, since non-conformant packets may still be forwarded, but with rate $\rho$ (once the bucket is empty). The shaping mechanism is typically obtained by placing a buffer of capacity $B_i$ before the TBF, to queue packets that cannot be immediately forwarded because of lack of tokens. The TBF, hence, governs the rate at which packets are actually extracted from the data buffer and passed to the output (link) buffer. The size $B_i$ of the input buffer determines the flexibility left to the source to temporarily violate the SLA thresholds: once the data buffer is full, the input traffic must drop below $\rho$ to avoid dropping. A graphical representation of this case (which generalizes the soft shaper one) is given in Fig. 0.5.

The chosen policy determines how the TB handles congestion and interacts with other traffic management mechanisms. By controlling the rate and burstiness of the output traffic, the TB ensures that traffic flows conform to the desired rate profile, preventing excessive bursts and maintaining network stability.

When the TBF is used as a shaper, the long-term average output rate is limited by the token generation rate $\rho \leq \mu$. The capacity the parameter $b$, instead, determines the maximum duration of the burst period, $\tau$, when the source transmits at peak rate $\mu$. This is given by:

$$\tau = \frac{b}{\mu - \rho}\,.$$

#### *0.3.0.1 Shaping sources with only rate constraints*

Consider then a bursty source characterized by the following parameters:

- *mean traffic rate*: $\bar{\mathrm{G}} = \lim_{t\to\infty} \frac{1}{t} \int_0^t \mathrm{G}(u)\mathrm{d}u$ , where $\mathrm{G}(t)$ is the traffic generated by the source at time $t$;

- *peak traffic rate*: $\mathrm{G}^* = \max_t\{\mathrm{G}(t)\}$;

- *maximum burst duration*: $T_{\text{burst}} = \max\{\delta : \mathrm{G}(t) = \mathrm{G}^*,\ \forall t_0, \forall t \in [t_0, t_0 + \delta]\}$.

To sustain the traffic generated by such a source, the SLA shall specify the following values for the associated TBF:

$$\rho = \frac{\bar{\mathrm{G}}}{D}\ ; \quad \mu = \frac{\mathrm{G}^*}{D}\ ; \quad b = T_{\text{burst}}(\mu - \rho)\ ;$$

where $D$ indicates how many bits can be transmitted per token.

To be noticed, that if the source transmits at a rate $g = G(t)/D$, larger than $\rho$ but lower than $\mu$, the actual time to empty the buffer will be longer than $T_{\text{burst}}$. For example, a full bucket would empty in a time

$$T_{\text{empty}}(g) = \frac{b}{g - \rho} > T_{\text{burst}}\ .$$

The maximum burst duration, indeed, just limits the time at which the source is allowed to transmit at its declared peak rate. Temporary oscillations above and below the average rate $\bar{\mathrm{G}}$ do not generate proper bursts. Clearly, in the reality, there might be intermediate situations, where some sources may generate traffic at a considerably higher rate than their average, but still significantly lower than the peak rate. In this case, the TBF model is not longer a suitable Policer/Shaper mechanism, and more sophisticated algorithms shall be used. In the following we limit our attention to sources that behave according to the simple "normal-burst" model, so that they either transmit at a rate that is close to their average rate, or at their peak rate. Moreover, for the sake of simplicity, we will assume $D = 1$.

#### *0.3.0.2 Shaping sources with rate and delay constraints*

If the traffic source also specifies delay constraints $d_{\max}$, the SLA must include some additional parameters, namely the capacity $c$ of the output link, and the size of the input buffer $B_i$. The size $B_o$ of the output buffer has also to be properly dimensioned to avoid undesired buffer overflows. In the following we derive the maximum buffers level and packet delay when the source transmit at its peak rate $G^* = \mu$ for a prolonged time.

First of all, we observe that the link rate has to satisfy the constraints $\rho < c$, otherwise the link would not even been able to sustain the long-term average traffic of the source, and the system will systematically drop packets in steady state. In the following we analyze the behavior of the level of the input and output buffer in time, assuming that the bucket is full of tokens at time zero. We distinguish two cases, namely $c \geq \mu$ and $\rho < c < \mu$.

*Case 1: $c \geq \mu$.* If the link rate larger than or equal toTBF peak rate, the output queue is certainly stable and the output buffer is empty. The input buffer will remain empty as long as there are tokens in the bucket, i.e., up to time $T_1 = T_{\text{burst}} = b/(\mu - \rho)$. After this time, the input buffer starts filling up at rate $\mathrm{G}^* - \rho = \mu - \rho$, reaching its maximum after an additional time $T_2 = \frac{B_i}{\mu - \rho}$. Any non-dropped packets generated after this time will suffer the maximum delay, given by the time taken by TBF to transmit all the bits in the input buffer before the packet, which is equal to

$$d_{\max} = \frac{B_i}{\rho}.$$

Inverting this equation we easily obtained the input buffer size to guarantee the delay remains below the threshold:

$$B_i = \rho d_{\max}\ .$$

*Case 2:* $c < \mu$. If $\rho < c < \mu$, the output buffer will fill up during burst periods, introducing a certain delay in packet forwarding. The maximum length of the output queue is reached at the end of the burst period, when the link input rate $\lambda(t)$ drops to $\rho$ because of the lack of tokens in the bucket. The maximum level reached by the output buffer is hence equal to

$$B_o = T_{\mathrm{burst}}(\mu - c) = b\frac{\mu - c}{\mu - \rho}\,. \tag{0.2}$$

This setting guarantees that no packet is dropped because of lack of buffer space in the output link, *irrespective of the dynamic of the input flow*. In fact, after a burst of maximum duration, the rate $\lambda(t)$ cannot exceed $\rho$ again until a few tokens accumulate in the bucket. The tokens collected over a time window $\tau$ during which $\lambda(t) \leq \rho$ will be equal to $\Delta_t = \int_0^\tau (\rho - \lambda(t))\mathrm{d}t$. However, in this period the link buffer will empty of $\Delta_o = \int_0^\tau (c - \lambda(t))\mathrm{d}t$ bits. Since $c > \rho$, then $\Delta_o > \Delta_t$, therefore there is always sufficient space in the output buffer to hold the next burst of packets.

We now focus on the input buffer. As long as $\lambda(t) = \mu$, the input buffer remains empty. After a period $T_1 = T_{\mathrm{burst}}$, however, the output rate drops to $\lambda(t) = \rho$ and, if the source keeps transmitting at the peak rate, the excess traffic is queued in the input buffer. The input buffer that starts increasing linearly in time, with slope $s_i = \mu - \rho$, while the level of the output buffer decreases with slope $s_o = c - \rho$. We hence set to $T_1$ our temporal reference point.

Let $T_i = B_i/(\mu - \rho)$ be the time needed to saturate the input buffer $B_i$. Similarly, let $T_o = B_o/(c - \rho)$ be the time required to empty the output buffer.

Now, consider a packet generated at any time $t = T_1 + \tau$, with $\tau < T_i$. It will experience a delay given by

$$d(t) = d_i(t) + d_o(t + d_i(t)) \tag{0.3}$$

where

$$d_i(t) = \frac{b_i(t)}{\rho} = \frac{(\mu - \rho)\tau}{\rho} \tag{0.4}$$

and

$$d_o(t + d_i(t)) = \frac{b_i(t + d_i(t))}{c}\,. \tag{0.5}$$

Eq. (0.4) gives the time taken by the packet to reach the head of queue in the input buffer and be forwarded to the output buffer, which is hence reached at time $t + d_i(t)$. The tagged packet finds a backlog of $b_i(t + d_i(t))$ at the output queue, which is served in a time $\frac{b_i(t+d_i(t))}{c}$. Now, if $\tau + d_i(t) \geq T_o$, then $b_i(t + d_i(t)) = 0$ and $d_o(t + d_i(t)) = 0$. Conversely, if $\tau + d_i(t) < T_o$, we have

$$d_o(t + d_i(t)) = \frac{B_o - (c - \rho)(\tau + d_i(t))}{c} = \frac{B_o}{c} - \tau\left(1 - \frac{\rho}{c}\right) + \frac{\rho}{c}d_i(t) - d_i(t)\,.$$

Replacing this expression in (0.3), and then $d_i(t)$ with (0.4), we obtain

$$d(t) = \frac{B_o}{c} + \tau\left(\frac{\mu}{c} - 1\right)\,. \tag{0.6}$$

Since $\mu > c$, the total delay $d(t)$ grows linearly in time, after $T_1$. The maximum is hence reached at time $t = T_1 + T_i$, when the input buffer reaches saturation. The maximum delay

is hence given by

$$\begin{aligned} d_{\max} &= \max\left\{\frac{B_i}{\rho}, \frac{B_o}{c} + T_i\frac{\mu - c}{c}\right\} \\ &= \max\left\{\frac{B_i}{\rho}, (b + B_i)\frac{\mu - c}{c(\mu - \rho)}\right\}. \end{aligned}$$

If the maximum of the above equation is reach by the first terms, we have $B_i = d_{\max}\rho$. Otherwise, $B_i = b - c\frac{\mu-\rho}{\mu-c}d_{\max}$. We can hence conclude that the value of the input buffer that guarantees the delay does not exceed the maximum is

$$B_i = \max\left\{b - c\frac{\mu - \rho}{\mu - c}, \rho\right\}d_{\max}.$$

Finding the value of $c$ for which the input buffer size exceeds $\rho d_{\max}$ is left as an exercise to the reader.